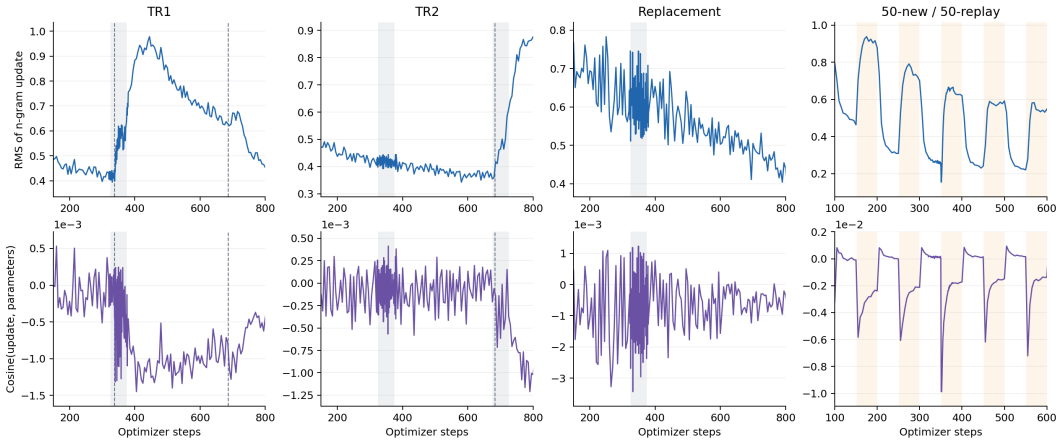


Figure 13: Two representative n-gram observables. The top row shows the RMS magnitude of the RMSProp update vector; the bottom row shows its cosine similarity with the parameter vector. Columns correspond to the four controls. Shading and boundaries follow Figure 12. Each panel uses its own visible vertical range. Unsmoothed curves are used to compare event timing and shape.

B.3 Selecting the controlled optimization setting

After localizing the n-gram pathway, we rebuilt the phenomenon in the vanilla architecture of Section 2.3. The optimization settings were selected to fit the training data and expose a readily measurable replay gap within a short experimental budget. The gap was inspected during selection; validation loss was not the selection objective. We distinguish the original selection screens from subsequent sensitivity checks below.

Shared protocol. The screens use an 8-layer, 6-head, 768-dimensional Transformer with learned absolute positions, LayerNorm, tied token embeddings, vocabulary size 8192, and sequence length 2048. A single clean $2^{20} \times 768$ table for each of bigrams and trigrams is added at the input, without gating or multi-hash concatenation. The training corpus is shard 1, replayed in a fixed order with data seed 42; validation uses four fixed batches from the disjoint shards 2–10 and 6542. Each update contains 72 sequences (147,456 prediction tokens). Computation uses BF16 autocast without compilation. Unless stated otherwise, experiments use seed 42 and record online training loss before the update and fixed-validation loss after it every 10 steps. All losses and gaps below are in nats/token.

The backbone uses AdamW with $(\beta_1, \beta_2) = (0.8, 0.95)$ and matrix weight decay 0.1, excluding the token and position embeddings and the tied output embedding. The tables use momentum-free, bias-corrected RMSProp with $\beta_2 = 0.99$, $\epsilon = 10^{-10}$, and no weight decay. Writing s_T for the table multiplier, the learning rates are

$$\eta_B(t) = \eta_B^{\max} w(t), \qquad \eta_T(t) = s_T \eta_B(t), \qquad w(t) = 0.25 + 0.75 \min\left\{\frac{t-1}{99}, 1\right\}, \quad t \geq 1. \tag{15}$$

Both rates reach their maxima at step 100 and remain constant thereafter, so later replay boundaries do not coincide with learning-rate changes. The selected maxima are $\eta_B^{\max} = 0.0006$ and $\eta_T^{\max} = 128\eta_B^{\max} = 0.0768$. The multiplier specifies nominal learning rates; adaptive normalization means it is not a statement that table parameters move 128 times as far as backbone parameters.

B.3.1 Why a backbone learning rate of 6×10^{-4}?

The initial clean-table screen compared 4×10^{-4}, 6×10^{-4}, and 4×10^{-3} with $s_T = 2$ (Table 2). The largest rate gave poor training loss and little separation. At 6×10^{-4}, the model fitted the replayed training data more strongly than at 4×10^{-4} and produced a larger gap within 1000 steps. We therefore retained 6×10^{-4} for the subsequent table-rate screen. The lower validation loss at 4×10^{-4} makes clear that this was a choice of an informative diagnostic regime, not a validation-

optimal rate. Because s_T was fixed, the absolute table rate also changed in this initial screen; it does not isolate a backbone-only causal effect.

Table 2: Initial rate selection with table multiplier $s_T = 2$, seed 42, step 1000. Both absolute rates change across rows.

$\eta_B^{\max}$	$\eta_T^{\max}$	Train	Validation	Gap
0.0004	0.0008	3.121	4.309	1.187
0.0006	0.0012	2.899	4.433	1.534
0.004	0.008	6.174	6.234	0.060

A later check varied only the backbone rate while holding the entire table-rate trajectory fixed: $\eta_T^{\max} = 0.0768$ and $s_T = 0.0768/\eta_B^{\max}$. Each rate has a matched no-gram control. Table 3 compares the same step, 2000, in every run, despite their different final budgets. Excess gaps remain large from 6×10^{-5} through 2×10^{-3}. At 4×10^{-3}, the small gap accompanies poor fitting: input-model train/validation losses are $6.694/6.741$, and the no-gram losses are $6.379/6.393$. Thus the disappearance of the gap at this rate does not demonstrate successful generalization with a well-trained memory. The reference rate lies inside a broad regime exhibiting forking; the evidence does not single it out as a universal optimum.

Table 3: Backbone-rate sensitivity with $\eta_T^{\max} = 0.0768$ fixed. Seed 42, raw step-2000 records; this diagnostic family evaluates every 50 steps. Train and validation columns describe the input model. Net gap subtracts the matched no-gram gap.

$\eta_B^{\max}$	Train	Validation	Input gap	No-gram gap	Net gap
0.00006	2.475	5.522	3.047	0.057	2.991
0.0001	2.151	5.733	3.582	0.073	3.508
0.0003	1.545	6.411	4.866	0.163	4.702
0.0006	1.251	6.940	5.689	0.255	5.434
0.001	1.243	7.269	6.026	0.195	5.831
0.002	1.487	7.332	5.844	0.075	5.770
0.004	6.694	6.741	0.046	0.014	0.032

B.3.2 Why a table learning-rate multiplier of 128?

We then fixed $\eta_B^{\max} = 6 \times 10^{-4}$ and scanned s_T from 0.5 to 1024 with RMSProp $(0, 0.99)$ (Table 4). At step 1000 the gap increases from 0.465 at $0.5\times$ to approximately 2.6–2.7 over $16\times$–$256\times$. The archived selection curve reaches 2.732 at $128\times$. We adopted this rate to expose the failure clearly within 1000 steps and held it fixed in the main experiments. This rationale explicitly includes selection on the observed gap.

The high-gap region is broad. At step 2000, all tested scales from $8\times$ through $1024\times$ yield gaps between 5.335 and 6.020; $128\times$ is not the largest. Separately trained 2000-step runs at $64\times$ and $128\times$ have step-1000 gaps 2.742 and 2.731, respectively, reversing their small ordering in the selection curve. These results support a range of settings that express forking, rather than a uniquely optimal multiplier. Large nominal table rates affect the magnitude and timing of the gap, but the phenomenon is also present at much smaller rates: the $2\times$ RMSProp runs at seeds 42–44 have step-1000 gap 1.530 ± 0.025 (mean $\pm$ sample standard deviation).

B.3.3 Optimizer and schedule checks

The table optimizer was kept simple: adaptive normalization without a first moment or weight decay. At $s_T = 2$, the corrected β_2 sweep over $0.95, 0.98, 0.99, 0.995, 0.999$ gives step-1000 gaps $1.239, 1.460, 1.551, 1.594, 1.669$, respectively (seed 42). The selected 0.99 is not the gap-maximizing value. Changing it to 0.999 also preserves large 2000-step gaps at $8\times$ and $128\times$ (5.899 and 5.399). These are local sensitivity checks, not evidence that the second-moment decay has no effect.

At $2\times$, an AdamW table-update implementation with $(\beta_1, \beta_2) = (0, 0.99)$ gives gap 1.521 ± 0.016 across seeds 42–44, close to RMSProp's 1.530 ± 0.025. With zero first moment and no table weight

Table 4: Table-rate selection and extended-budget sensitivity. Backbone rate 0.0006, RMSProp $(0, 0.99)$, seed 42. Train, validation, and G_{1000} use the archived selection curve. For $32\times$, $64\times$, and $128\times$, G_{2000} comes from a separate longer run; for the other extended scales, both readouts come from the same run. Dashes denote no 2000-step result in this scan.

s_T	$\eta_T^{\max}$	Train$_{1000}$	Validation$_{1000}$	G_{1000}	G_{2000}
0.5	0.0003	3.521	3.986	0.465	–
1	0.0006	3.265	4.123	0.858	–
2	0.0012	2.883	4.433	1.551	–
3	0.0018	2.759	4.645	1.886	–
4	0.0024	2.757	4.830	2.072	–
8	0.0048	2.652	5.080	2.428	6.020
16	0.0096	2.689	5.328	2.639	5.761
32	0.0192	2.637	5.284	2.647	5.659
64	0.0384	2.572	5.278	2.706	5.740
128	0.0768	2.547	5.279	2.732	5.724
256	0.1536	2.621	5.299	2.677	5.543
512	0.3072	2.669	5.240	2.571	5.474
1024	0.6144	2.805	5.255	2.450	5.335

decay, these two implementations use essentially the same normalized-gradient update, differing in the denominator epsilon; their agreement is an implementation check, not a comparison with conventional momentum-based AdamW. At $128\times$, their seed-42 gaps are likewise 2.731 and 2.727. Momentum-free SGD at the same nominal rates produces much smaller gaps (0.054 ± 0.025 at $2\times$ and 0.047 at $128\times$), so the nominal multiplier cannot be interpreted independently of the optimizer.

Finally, the warmup-start check retained the 100-step duration and varied only its initial multiplier. Starts at 0.1, 0.25, and 0.5 give step-1000 gaps 1.504, 1.551, and 1.442, with training losses 2.905, 2.883, and 3.004 (seed 42, $s_T = 2$). Starting at the full rate without warmup gives training loss 4.032 and gap 0.534. The intermediate start 0.25 is therefore not an isolated point, while removing warmup changes the optimization trajectory substantially. We use the same warmup-constant schedule throughout the main controlled comparisons.

Scope and run provenance. The rate screens are single-seed unless a three-seed statistic is explicitly reported; they do not constitute a factorial search across all optimizer choices. The initial screen uses `lrscan_input_lr*_wc_fixed`. Table-rate records use `optv5c_rms_b099_s*_fixed` (including the completed $2\times$ retry) and `optv5f_rms_b099_s*_fixed`; extended budgets add `_2k` before `_fixed`. The fixed-absolute-rate controls use `blrabs_{input,nogram}_lr*_tlr0p0768_*_fixed`. Optimizer, decay, and warmup checks use the `optv5c`, `optv5f`, `optv5g`, and `optv5h` families. The source supplement `supplement/parameter_selection_manifest.json` maps each reported entry to its exact run ID, seed, step, configuration, and source-file hashes. Historical multi-hash discovery configurations and failed or incomplete runs do not supply the numerical evidence in this subsection.

C Standard Setting and Measurement Conventions

The choice of backbone learning rate, table learning-rate multiplier, optimizer, and warmup schedule is documented with selection and sensitivity experiments in Appendix B.3.

C.1 Loss and frequency measurements

The discovery-stage configuration in Appendix B.1 and the controlled setting in Section 2.3 should be distinguished. They use different optimizers and injection implementations. Frequencies $F(i)$ refer to counts in a single unrepeated training corpus, even when that corpus is consumed many times.

For online curves, the training loss is obtained from the training forward pass before the corresponding update, while validation can be evaluated after that update. A fixed-probe gap instead compares fixed